



TF

TASKFLOW RH

Piloter les projets RH, suivre les tâches, décider.

Application React de gestion des activités du département des ressources humaines.

Présentation professionnelle • 2026

Gael Ekofo | React • Vite • JSON Server

Pourquoi TaskFlow RH ?

Une interface unique pour transformer les activités RH en projets mesurables.

Le besoin

- Centraliser les projets du département.
- Suivre l'état réel de chaque tâche.
- Identifier rapidement les urgences.
- Donner une vision globale au responsable.

La réponse

TaskFlow RH

Projets + tâches + rôles + statistiques

L'application relie chaque tâche à un projet, calcule la progression et adapte les données affichées au rôle connecté.

Une visibilité adaptée à chaque rôle

Le Responsable RH supervise le département ; les autres utilisateurs voient leurs activités.

RESPONSABLE RH

Vue globale

Tous les projets, toutes les tâches et les statistiques du département.

GESTIONNAIRE RH

Vue personnelle

Projets créés par le gestionnaire et tâches associées.

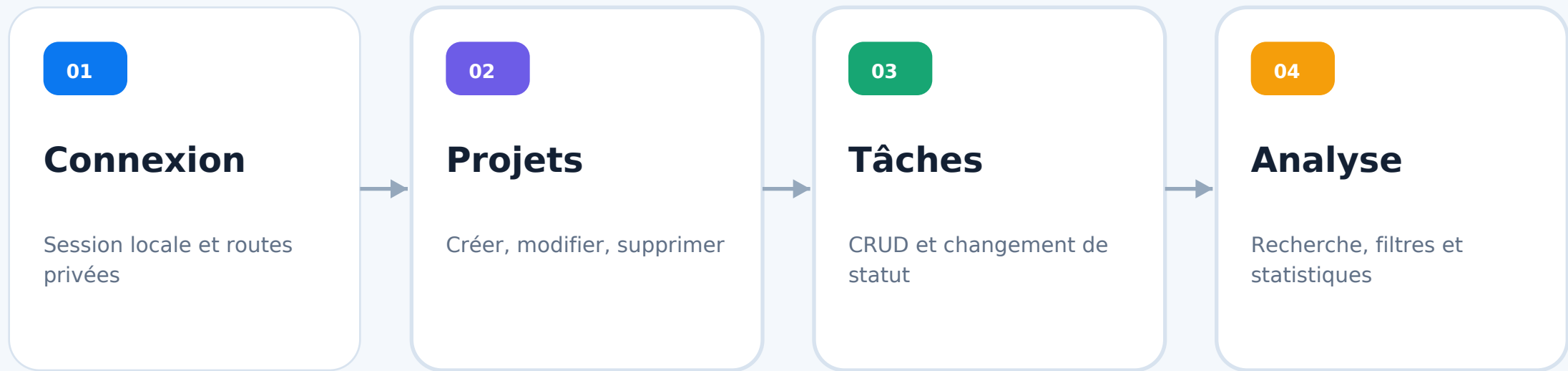
AGENT RH

Vue personnelle

Projets créés par l'agent et tâches associées.

Un parcours complet

De la connexion au pilotage, chaque étape est simple et cohérente.



Comment les éléments communiquent

React gère l'interface ; les fonctions API dialoguent avec JSON Server.



RÈGLE CENTRALE

Responsable RH → tous les projets | Autres rôles → projets personnels

Une structure React claire

Chaque dossier possède une responsabilité précise.

ARBORESCENCE

```
src/
├─ api/          # Requêtes HTTP
├─ components/   # Éléments réutilisables
├─ context/      # Contexte React
├─ hooks/        # Hooks personnalisés
├─ pages/        # Écrans
└─ utils/        # Statistiques

db.json          # Données simulées
```

api/

Accès aux projets, tâches et dashboard

components/

Navigation, cartes, routes et modales

context + hooks

Authentification séparée

utils/

Calculs indépendants de l'interface

Afficher les projets selon le rôle

La fonction adapte automatiquement l'adresse interrogée.

```
src/api/projets.js

export async function obtenirProjets(
  utilisateurId,
  roleUtilisateur
) {
  if (!utilisateurId) return [];

  const adresseProjets =
    roleUtilisateur === "Responsable RH"
      ? API_URL + "/projets"
      : API_URL + "/projets?utilisateurId="
        + encodeURIComponent(utilisateurId);

  const reponse = await fetch(adresseProjets);
  if (!reponse.ok) {
    throw new Error("Chargement impossible.");
  }
  return reponse.json();
}
```

Explication

- La fonction reçoit le rôle.
- Le responsable utilise /projets.
- Les autres utilisent utilisateurId.
- Les erreurs HTTP sont interceptées.

Charger les données dans Projets.jsx

L'identifiant et le rôle sont transmis à l'API.

```
src/pages/Projets.jsx

const utilisateurId = utilisateur?.id;
const roleUtilisateur = utilisateur?.role;

const chargerDonnees = useCallback(async () => {
  const [projetsRecus, tachesRecues] =
    await Promise.all([
      obtenirProjets(
        utilisateurId,
        roleUtilisateur
      ),
      obtenirTaches(),
    ]);

  setProjets(projetsRecus);
  setTaches(tachesRecues);
}, [utilisateurId, roleUtilisateur]);
```

Pourquoi ce code ?

Promise.all

Charge deux ressources en parallèle.

useCallback

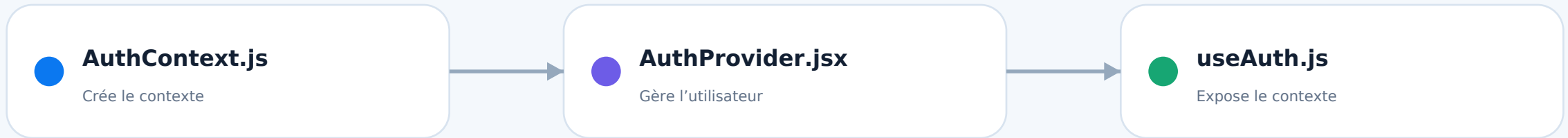
Stabilise la fonction de chargement.

Dépendances

Recharge si l'utilisateur ou le rôle change.

Séparer contexte, fournisseur et hook

Cette organisation clarifie les responsabilités et respecte Fast Refresh.



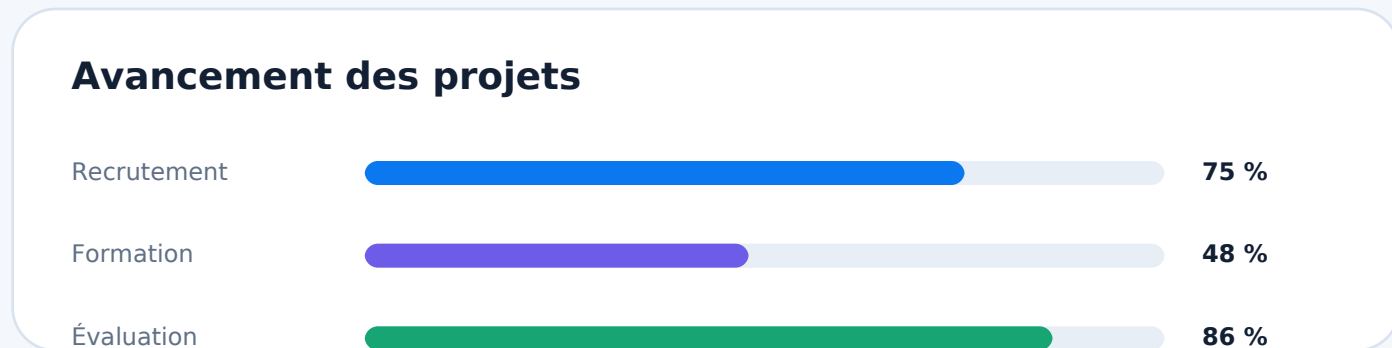
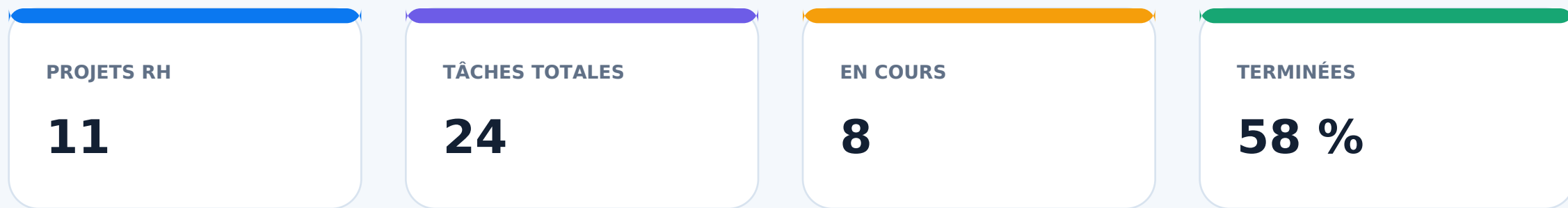
```
src/hooks/useAuth.js

import { useContext } from "react";
import { AuthContext } from "../context/AuthContext";

export function useAuth() {
  const contexte = useContext(AuthContext);
```

Des données transformées en décisions

Les indicateurs proviennent des tâches liées aux projets visibles.



Tâches urgentes

- Publier une offre
- Valider les candidatures
- Préparer les entretiens

Trois collections reliées

JSON Server simule une API REST pour l'apprentissage.



Démarrer en quelques commandes

Deux terminaux : un pour l'API et un pour React.

TERMINAL

```
# Installation
git clone https://github.com/Softcompute/taskflow-rh.git
cd taskflow-rh
npm install

# Terminal 1
npm run api

# Terminal 2
npm run dev
```

Adresses locales

Interface React

<http://localhost:5173>

API JSON Server

<http://localhost:3000>

Collection projets

localhost:3000/projets

Un projet vérifié avant livraison

Oxlint contrôle le code et Vite valide la compilation.

Contrôle du code

● ● ● RÉSULTAT

```
$ npm run lint  
Found 0 warnings and 0 errors.
```

Compilation production

● ● ● RÉSULTAT

```
$ npm run build  
✓ 45 modules transformed.  
✓ built in 11.72s
```

✓ Code propre • ✓ Build réussi • ✓ Prêt pour Git

Une base pédagogique

Pour de vraies données RH, un backend sécurisé est indispensable.

Limites actuelles

- Mots de passe non hachés.
- Permissions contrôlées dans React.
- Pas de vraie session serveur.
- Pas de données RH confidentielles.

Évolutions recommandées

01 Backend Node.js / Express

02 Base MySQL ou PostgreSQL

03 Mots de passe hachés + JWT

04 Permissions côté serveur

The logo consists of a blue rounded square containing the white letters 'TF' in a bold, sans-serif font.

CONCLUSION

TaskFlow RH rend le travail du département visible et mesurable.

Authentification, rôles, projets, tâches, recherche, filtres, statistiques, responsive et documentation.

github.com/Softcompute/taskflow-rh